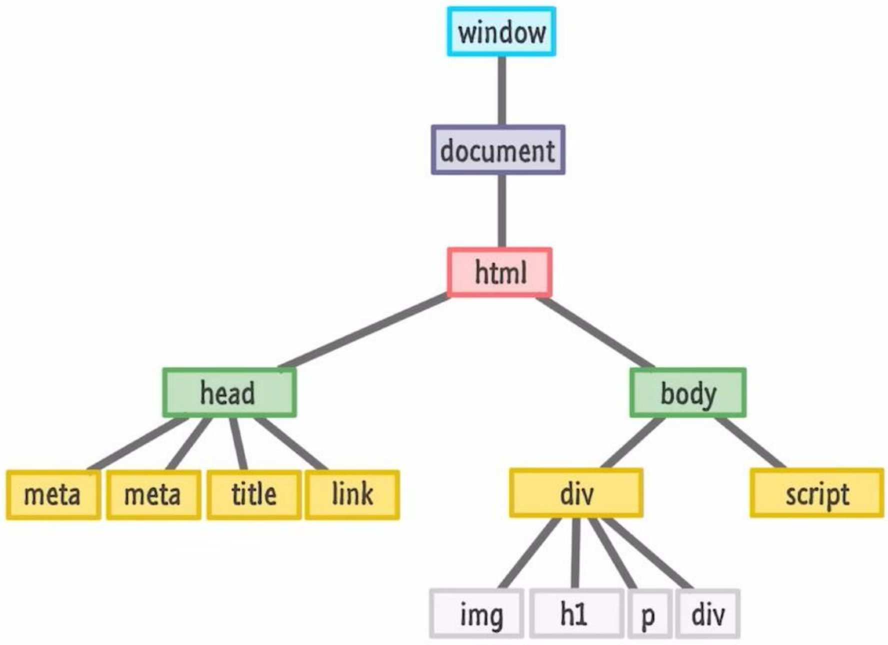
Understanding the DOM

Document Object Model

The DOM is a programming interface for web documents. It represents the page so that programs can change the document structure, style, and content.

- **Tree Structure:** Represents the document as a tree of nodes.
- **Dynamic Access:** Allows JavaScript to manipulate HTML and CSS.
- **Language Neutral:** While often used with JS, it's designed to be independent.
- **Live Representation:** Updates in real-time as the document is modified.



DOM Manipulation Techniques

1. Selecting by ID

Target a unique element using its ID attribute. Fast and efficient.

```
document.getElementById('my-header');
```

2. Selecting by Class Name

Retrieve a collection of all elements that share a specific class.

```
document.getElementsByClassName('item-list');
```

3. Selecting by Tag Name

Select all elements of a certain type, such as all <p> or <div> tags.

```
document.getElementsByTagName('div');
```

DOM manipulation

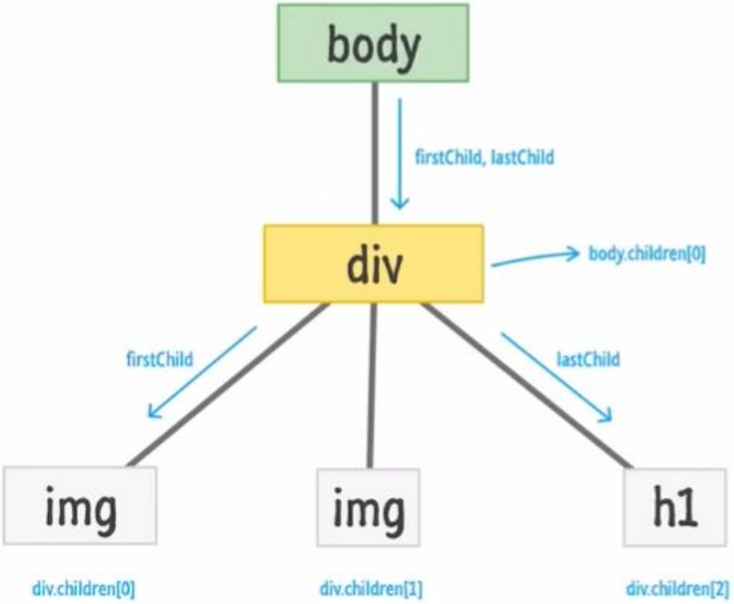
Properties

- **tagName:** Returns the tag name for element nodes.
- **innerText:** Retrieves the visible text content of the element and all its descendants.
- **innerHTML:** Returns the HTML or plain text contents contained within the element.
- **textContent:** Provides the textual content, including text from hidden elements.

DOM Tree Structure

Core Node Types

- **Element Nodes:** Represent HTML elements like <div>, <h1>, and <img>, which form the backbone of the document structure.
- **Text Nodes:** Contain the actual text content found within elements
- **Comment Nodes:** Represent the source code comments within the HTML that are part of the DOM but not displayed on the page.



Question 1: DOM Exercise

Part A: Element Creation

Create an `<h1>` heading element.

Initial text content:

`"Welcome to Coding World"`

Part B: DOM Manipulation

Using JavaScript, **append** the following string to the existing heading:

`" - Learn, Practice & Grow"`

Tip: Consider using .innerText or .textContent properties.

Task: Div Creation & Styling

Objective: Create five `div` elements, each with the class `box`.

JavaScript Implementation Steps:

1. Sequential Labeling

Label each div from `"Box 1"` to `"Box 5"`.

2. Odd Styling

Apply a **red border** to all odd-numbered boxes.

3. Even Styling

Apply a **blue border** to all even-numbered boxes.